

9. Thread

동시성과 효율성을 위한 필수 개념

쓰레드란 무엇인가요?

- 쓰레드(Thread)는 '실행의 흐름'
 - 하나의 프로세스 안에서 여러 작업을 동시에 실행하는 가장 작은 단위
 - 하나의 회사(프로세스) 안에서 여러 팀원(쓰레드)이 각자의 업무를 처리하는 것'

1. 프로세스(Process)란?

- **정의:** 운영체제로부터 독립적인 메모리 공간과 자원(파일, 코드 등)을 할당 받아 실행 중인 프로그램입니다.
- **비유:** 우리가 컴퓨터에서 실행하는 하나의 **응용 프로그램**이라고 생각하면 쉽습니다. 예를 들어, 크롬 웹 브라우저, 워드 프로그램, 게임 등이 각각 하나의 프로세스가 될 수 있습니다.
- **특징:**
 - 완전히 독립적인 주소 공간을 가집니다. 다른 프로세스의 메모리에는 접근할 수 없습니다.
 - 새로운 프로세스를 생성하는 데는 많은 시스템 자원과 시간이 소요됩니다.

2. 쓰레드(Thread)란?

- **정의:** 프로세스 안에서 실제로 작업을 수행하는 가장 작은 실행 단위입니다.
- **비유:** 크롬 웹 브라우저라는 프로세스 안에서, 여러 개의 탭을 열어 동시에 인터넷 서핑을 하거나, 파일을 다운로드하고, 영상을 재생하는 것과 같습니다. 여기서 각 탭을 처리하는 작업 흐름이 바로 쓰레드라고 볼 수 있습니다.
- **특징:**
 - 같은 프로세스에 속한 다른 쓰레드들과 **메모리(코드, 데이터, 힙)를 공유**합니다.
 - 각 쓰레드는 고유의 실행 스택(Stack), 프로그램 카운터(Program Counter), 레지스터(Register)를 가집니다.
 - 프로세스보다 가볍고, 생성 및 컨텍스트 스위칭(Context Switching: 쓰레드 간 제어권 전환)에 드는 비용이 적습니다.

3. 쓰레드를 왜 사용할까요? (멀티쓰레딩의 장점)

- **자원 공유의 용이성:** 같은 프로세스 내에서 데이터 교환이 쉬움
 - 같은 프로세스 내의 쓰레드는 메모리를 공유하므로, 별도의 통신 메커니즘(IPC: Inter-Process Communication) 없이도 데이터를 쉽게 주고받을 수 있습니다.
- **응답성 향상:** 사용자가 멈춤 없는 부드러운 서비스를 경험하게 함
 - 웹 브라우저 예시처럼, 시간이 오래 걸리는 작업을 쓰레드에 맡기면 메인 쓰레드는 사용자의 입력이나 UI를 처리하는 데 집중할 수 있습니다. 덕분에 프로그램이 멈추거나 '먹통'이 되는 현상을 막을 수 있습니다.
- **효율적인 자원 활용:** 멀티코어 CPU를 효과적으로 활용하여 전체 처리량 증가
 - 여러 작업을 동시에 처리함으로써 CPU의 유휴 시간을 줄이고, 멀티코어 시스템에서 각 코어를 효율적으로 활용하여 전체적인 처리량을 높일 수 있습니다.
- **오버헤드 감소:** 새로운 프로세스를 생성하는 것보다 쓰레드를 생성하는 것이 훨씬 가볍고 빠릅니다.

4. 쓰레드의 단점

- **동기화 문제:** 공유 자원에 여러 쓰레드의 동시접근으로 데이터가 꼬임
 - 여러 쓰레드가 하나의 메모리 공간을 공유하기 때문에, 동시에 같은 데이터에 접근하여 값을 변경할 경우 예상치 못한 오류(데이터 불일치)가 발생할 수 있습니다. 이를 해결하기 위해 **동기화(Synchronization)** 메커니즘(뮤텍스, 세마포어 등)이 필요합니다.
- **프로그램 복잡성 증가:** 쓰레드 간의 관계 관리로 코드 복잡도 증가
 - 쓰레드 간의 제어 흐름과 데이터 공유를 관리해야 하므로, 코드가 복잡해지고 버그를 찾기 어려워질 수 있습니다.
- **예외 처리의 어려움:**
 - 한 쓰레드에 문제가 생기면, 같은 메모리를 공유하는 다른 쓰레드나 전체 프로세스에 영향을 미칠 수 있습니다.

5. Thread 사용 예

1. 웹 서버

- 웹 서버는 수많은 사용자의 요청을 동시에 처리해야 합니다.
- **쓰레드가 없을 때:** 쓰레드가 없으면 한 번에 하나의 요청만 처리 가능 (응답 지연)
 - 한 사용자의 요청(예: 복잡한 데이터베이스 조회)을 처리하는 동안 다른 사용자의 요청은 대기해야 합니다. 서버는 한 번에 하나의 요청만 처리할 수 있어서, 사용자가 많아질수록 응답 시간이 느려지고 서버가 멈출 수도 있습니다.
- **쓰레드를 사용할 때:** 각 요청에 별도의 쓰레드 할당 → 수백, 수천 개의 요청을 동시에 처리 → 사용자에게 빠른 응답 제공
 - 사용자의 요청이 들어오면 새로운 쓰레드를 생성하거나 미리 만들어 둔 쓰레드 풀에서 쓰레드를 할당해줍니다. 각 쓰레드는 독립적으로 해당 사용자의 요청을 처리하므로, 서버는 수많은 사용자의 요청을 동시에 처리할 수 있게 됩니다. 이것이 우리가 웹사이트를 사용할 때 랙(Lag) 없이 빠르게 페이지를 전환하거나 데이터를 받아볼 수 있는 이유입니다.

5. Thread 사용 예

2. 게임

- 온라인 게임은 실시간으로 여러 작업을 처리해야 합니다.
- **쓰레드가 없을 때:**
 - 캐릭터의 이동, 배경 음악 재생, 적 AI(인공지능) 연산, 네트워크 통신 등 모든 작업이 순차적으로 이루어져야 합니다. 이 경우, 한 작업이 지연되면 전체 게임이 멈추거나 끊기는 현상이 발생합니다.
- **쓰레드를 사용할 때:**
 - **메인 쓰레드:** 사용자 입력 처리와 화면 렌더링을 담당합니다.
 - **네트워크 쓰레드:** 다른 플레이어와의 통신을 처리합니다.
 - **사운드 쓰레드:** 배경 음악이나 효과음을 재생합니다.
 - **AI 쓰레드:** NPC(Non-Player Character)의 동작을 계산합니다.
- 이렇게 각 작업을 별도의 쓰레드에 분산하면, 네트워크 통신이 지연되어도 게임 화면은 부드럽게 유지될 수 있습니다. 덕분에 사용자들은 쾌적하게 게임을 즐길 수 있습니다.

5. Thread 사용 예

• 3. 모바일 앱(안드로이드/iOS)

- 모바일 앱은 사용자 인터페이스(UI)가 멈추지 않고 부드럽게 작동하는 것이 중요합니다.
- **쓰레드가 없을 때:** 네트워크 통신이나 큰 파일 다운로드 같은 시간이 오래 걸리는 작업을 ****메인 쓰레드(UI 쓰레드)****에서 직접 처리하면, 그 작업이 끝날 때까지 화면이 멈추고 버튼이 눌리지 않는 '앱 멈춤' 현상이 발생합니다.
- **쓰레드를 사용할 때:**
 - **메인 쓰레드:** 화면 그리기, 터치 이벤트 처리 등 UI 관련 작업만 담당합니다.
 - **백그라운드 쓰레드:** 네트워크 통신, 데이터베이스 접근, 파일 다운로드/업로드 등 시간이 오래 걸리는 작업을 맡습니다.
- 이처럼 작업을 분리하면, 사용자가 버튼을 눌러 데이터를 다운로드하는 동안에도 메인 쓰레드는 다른 UI 요소를 부드럽게 움직일 수 있습니다.
- 이 외에도 파일 압축, 동영상 인코딩, 빅데이터 처리 등 시간이 오래 걸리는 작업을 병렬로 처리할 때 쓰레드가 필수적으로 사용됩니다. 즉, ****"사용자에게 멈춤 없는 빠른 응답을 제공해야 하는 모든 서비스"****에서 쓰레드가 필요하다고 생각하시면 됩니다.

Python Threading

- threading 모듈은 파이썬에서 **멀티쓰레딩**을 구현하는 가장 기본적인 방법
 - 쓰레드를 생성하고, 실행하고, 관리하는 기능을 제공.
- 동시성 맛보기 예제
 - 두 개의 쓰레드가 각각 다른 작업을 동시에 수행
 - task1: '쓰레드 1: 5초 대기...'라는 메시지를 출력한 후 5초 동안 잠들기
 - task2: '쓰레드 2: 2초 대기...'라는 메시지를 출력한 후 2초 동안 잠들기
 - 만약 이 작업을 쓰레드 없이 순차적으로 실행하면 총 7초(5초 + 2초)가 걸리겠지만, **쓰레드를 사용하면 두 작업이 동시에 진행되므로 전체 실행 시간은 더 짧아집니다.**

```
import threading
import time

def task1():
    print("쓰레드 1: 5초 동안 대기합니다.")
    time.sleep(5)
    print("쓰레드 1: 작업 완료!")

def task2():
    print("쓰레드 2: 2초 동안 대기합니다.")
    time.sleep(2)
    print("쓰레드 2: 작업 완료!")

# 메인 프로그램 시작
print("메인: 두 쓰레드를 시작합니다.")
```

```
# 쓰레드 생성
t1 = threading.Thread(target=task1)
t2 = threading.Thread(target=task2)

# 쓰레드 시작
t1.start()
t2.start()

# 모든 쓰레드가 끝날 때까지 기다립니다.
# 이 코드가 없으면 메인 쓰레드가 먼저 끝나버릴 수 있습니다.
t1.join()
t2.join()

print("메인: 모든 쓰레드 작업이 완료되었습니다.")
```

- **예제 코드의 실행 결과**

- 메인: 두 쓰레드를 시작합니다.
- 쓰레드 1: 5초 동안 대기합니다.
- 쓰레드 2: 2초 동안 대기합니다.
- 쓰레드 2: 작업 완료!
- 쓰레드 1: 작업 완료!
- 메인: 모든 쓰레드 작업이 완료되었습니다.

- **결과 분석:**

- t1.start()와 t2.start()가 실행된 후,
두 쓰레드는 **동시에** 각자의 작업을 시작합니다.
- task2가 2초 후에 먼저 끝나고,
5초를 기다린 task1이 그 뒤에 끝납니다.
- t1.join()과 t2.join() 덕분에 메인 쓰레드는 두 보조 쓰레드가 모두 끝날 때까지
기다립니다

공유 자원 문제 예제: 동기화의 필요성

- 여러 스레드가 하나의 자원(변수, 파일 등)에 동시에 접근하면 **동기화 문제**가 발생할 수 있습니다. 아래 예제는 여러 스레드가 하나의 전역 변수를 동시에 증가시키려 할 때 발생하는 문제를 보여줍니다.

```
import threading
import time

shared_counter = 0

def increment_counter():
    global shared_counter
    for _ in range(100000):
        # 이 세 줄의 코드가 한 번에 실행되지 않고
        # 중간에 다른 쓰레드로 제어권이 넘어갈 수
        # 있습니다.
        current_value = shared_counter
        current_value += 1
        shared_counter = current_value
```

```
threads = []
for i in range(5):
    thread = threading.Thread
                        (target=increment_counter)
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()

print(f"최종 카운터 값: {shared_counter}")
```

- # 예상 값: 500000, 실제 값: 500000보다 작게 나옴

- **결과 분석:**

- 예상했던 최종 값은 500,000이지만, 실제로 실행해보면 대부분 500,000보다 작은 값이 출력됩니다.
- 그 이유는 `current_value = shared_counter`와 `shared_counter = current_value` 사이에서 ****경쟁 상태(Race Condition)****가 발생하기 때문입니다.
- 스레드 A가 `shared_counter`의 값을 읽은 후, 아직 값을 업데이트하기 전에 스레드 B가 먼저 값을 읽어버리면, 두 스레드 모두 같은 값을 가지고 작업을 시작하게 되어 한 번의 업데이트가 무시됩니다.

- 동기화 문제 해결: Lock 사용하기

- 이 문제를 해결하기 위해 threading.Lock을 사용해 공유 자원에 대한 접근을 잠그는(Lock) 방법을 사용할 수 있습니다.


```
import threading

shared_counter = 0
lock = threading.Lock() # Lock 객체 생성

def increment_counter_with_lock():
    global shared_counter

    for _ in range(1000000):
        lock.acquire()
        # 잠금: 다른 스레드는 이 블록에 들어올 수 없습니다.
        shared_counter += 1
        lock.release()
        # 잠금 해제: 이제 다른 스레드가 접근할 수 있습니다.
```

```
threads = []
for i in range(5):
    thread = threading.Thread(
        target=increment_counter_with_lock)
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()

print(f"최종 카운터 값 (Lock 사용):
      {shared_counter}")
```

Python Thread 요약

- `threading.Thread`로 스레드 생성
 - `.start()`로 실행
 - `.join()`으로 메인 스레드가 작업이 끝날 때까지 기다림
 - `lock.acquire()`: 진입 잠금
 - `lock.release()`: 작업 후 해제
-
- 스레드는 사용자에게 **멈춤 없는 빠른 응답**을 제공하는 모든 서비스의 핵심입니다.
 - 멀티코어 시대의 효율적인 병렬 프로그래밍을 위한 필수 개념입니다.